

MRLCO: 변화하는 MEC에서 빠르게 적응하는 DAG 오프로딩

Fast Adaptive Task Offloading in Edge Computing based on Meta Reinforcement Learning

Jin Wang, Jia Hu, Geyong Min, Albert Y. Zomaya, Nektarios Georgalas (2020)

메타 정책으로 시작하고, 새 환경에서는 적은 경험으로 task별 policy를 조정한다

환경이 바뀌면 기존 DRL policy가 다시 학습해야 한다

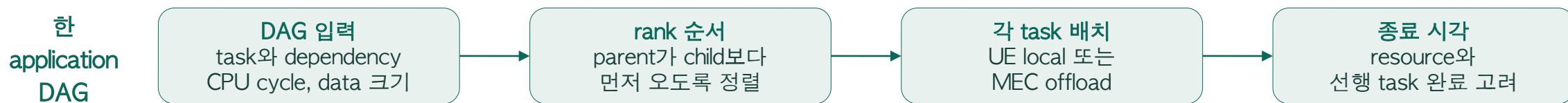
DAG 형태, task 수, 전송률이 달라지면 한 환경에 맞춘 policy의 표본 효율이 낮아진다

- | | | |
|----|-----------|--------------------------------------|
| 01 | 새 MEC 맥락 | DAG 또는 전송률이 달라진다 |
| ↓ | | |
| 02 | 일반 DRL | 새 data로 policy를 오래 조정한다 |
| ↓ | | |
| 03 | MRLCO의 준비 | 여러 맥락에서 meta 초기값을 학습한다 |
| ↓ | | |
| 04 | 새 맥락 적응 | 작은 sample과 update로 task별 policy를 만든다 |

목표는 매 환경의 즉시 정답이 아니라 새 환경에서 빠르게 유용해지는 초기 policy다

DAG 오프로딩은 의존성을 지키며 task를 배치하는 문제다

child task는 모든 parent task가 끝난 뒤에만 실행할 수 있다



순차 decoder가 결정을 내리더라도, 그 순서는 DAG의 dependency를 보존해야 한다

local과 offload의 latency는 같은 task에도 달라진다

offload는 uplink, VM 실행, downlink를 거치며 parent와 resource의 완료 시각도 기다린다

배치	계산 시간	통신 시간	종료 조건
UE local	$\text{CPU cycles} \div f_{\text{UE}}$	없음	parent 완료와 UE 가용 시각
MEC offload	$\text{CPU cycles} \div f_{\text{VM}}$	uplink + downlink	parent, 무선, VM 가용 시각
DAG 전체	각 task 종료 시각	dependency 간 data 이동	exit task 중 가장 늦은 종료

빠른 VM만으로 offload를 정할 수 없고, 통신과 선행 task 완료를 함께 계산해야 한다

강의용 계산 예시: 전송률이 바뀌면 선택도 바뀐다

아래 수치는 원문 실험값이 아니라 latency 모델을 재계산하는 예시다

01	parent A 완료	A가 4 ms에 local에서 끝난다
↓		
02	B를 local	20 ms 계산으로 B는 24 ms에 끝난다
↓		
03	B를 offload	3 ms uplink + 2 ms VM + 1 ms downlink
↓		
04	첫 조건의 선택	B는 10 ms에 끝나므로 offload가 빠르다
↓		
05	낮은 전송률	uplink가 25 ms면 32 ms가 되어 local이 빠르다

같은 DAG에서도 wireless condition이 달라지면 local과 offload의 상대 비용이 바뀐다

MRLCO의 MDP: 상태는 DAG와 지금까지의 배치다

한 learning task는 하나의 MEC 맥락에서 policy를 학습하는 MDP다

요소	논문의 정의	왜 필요한가
state	encoded DAG와 partial plan $A_{1:i}$	앞선 배치가 resource 가용 시각을 바꾼다
action	다음 task의 UE local 또는 MEC offload	두 실행 경로 중 하나를 선택한다
reward	결정 뒤 늘어난 end-to-end latency의 음수	짧은 전체 latency를 선호한다
learning task	DAG 집합 또는 rate 맥락의 MDP	여러 맥락에서 meta policy를 학습한다

computation task와 learning task를 구분해야 메타 학습의 대상이 선명해진다

seq2seq: DAG를 action 순서로 바꾸는 policy

encoder는 task embedding을 읽고, attention decoder는 이전 action을 참고해 다음 action을 만든다



decoder가 각 action을 따로 고르는 것이 아니라, 앞선 placement와 DAG context를 함께 본다

PPO inner loop: 새 MEC context 적응

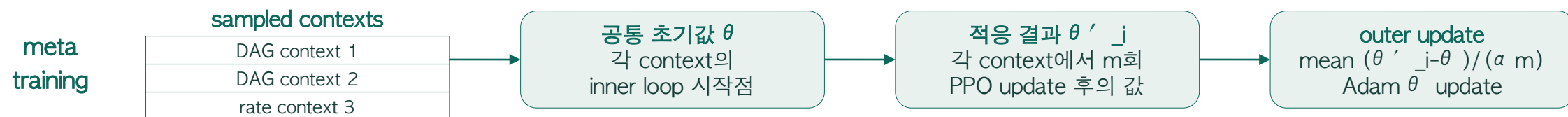
UE는 meta 초기값에서 시작해 local trajectory로 소수의 policy update를 수행한다

01	초기값	meta parameter θ 를 UE가 받는다
↓		
02	trajectory	sample policy로 DAG 배치 경험을 모은다
↓		
03	PPO 목적	clipped policy objective와 value loss를 계산한다
↓		
04	m회 update	Adam으로 task-specific θ_i 를 만든다
↓		
05	scheduler	학습된 policy로 DAG placement를 추론한다

inner loop는 새 learning task의 policy를 학습하는 단계이며, meta-training 자체가 아니다

first-order meta: 여러 context의 초기값 갱신

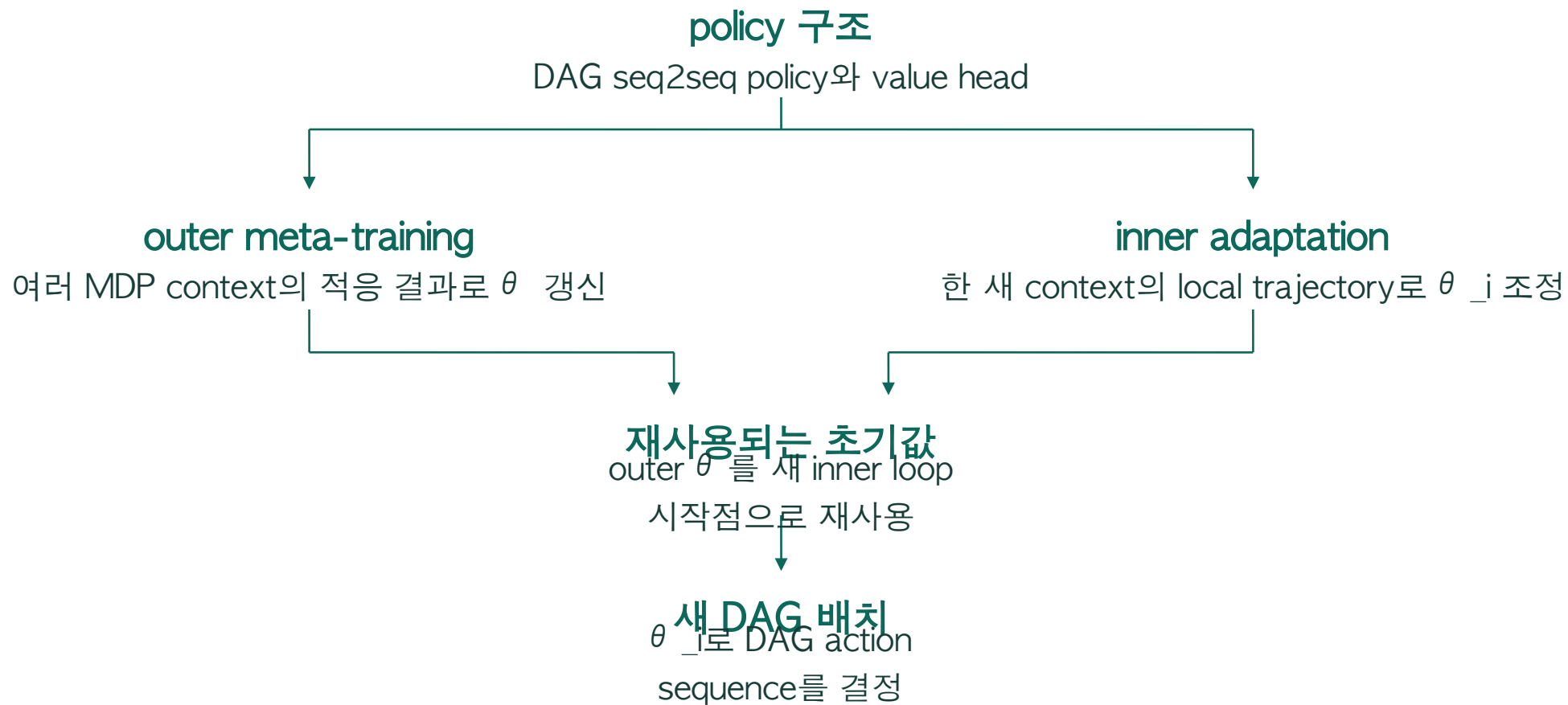
Eq. (22)는 sampled context의 $(\theta'_{-i} - \theta) / (\alpha m)$ 을 평균내어 θ 를 Adam으로 갱신한다



outer loop는 새 context의 빠른 적응을 위한 θ 를 학습한다

meta-training과 새 환경 적응의 차이

학습 대상은 둘 다 policy parameter이지만 입력 데이터와 목적이 다르다



평가 조건: synthetic DAG와 held-out learning task

결과는 이 simulator, update 수, test context에서만 읽어야 한다

항목	원문 조건	발표에서 유지할 한계
컴퓨팅	UE 1 GHz, VM 10 GHz	실제 단말의 training latency를 뜻하지 않는다
DAG	data 5-50 KB, work 10^7-10^8 cycles	synthetic workload 조건이다
학습	LSTM 2층, 256 units, inner m=3	특정 architecture와 hyperparameter다
test context	held-out topology, task number, rate	모든 MEC 환경 일반화 증명은 아니다

latency 수치와 adaptation 속도는 source가 정의한 synthetic MEC context에서의 비교 결과다

적응 결과: update 수와 test context

- Topology I에서는 MRLCO가 9회 update 뒤 HEFT-based 방법보다 낮은 latency를 보였다고 원문이 설명한다.
- Task 수 $n=30$ 의 그림에서는 한 번의 update 뒤 latency가 크게 낮아졌고, 20회 뒤 MRLCO가 표시된 baseline 중 가장 낮았다.
- 보지 못한 5.5, 8.5, 11.5 Mbps rate에서도 20회 update 뒤 MRLCO가 가장 낮은 latency를 보였다고 보고한다.
- 이 관찰은 held-out synthetic test context의 plot 결과이며, 보편적인 update 횟수 보장은 아니다.

fast adaptation은 적은 update 뒤 낮아지는 실험 결과다

Table 3 읽기: MRLCO도 optimal 값과는 차이가 남는다

아래는 held-out Topology I의 average latency 밀리초를 같은 표에서 비교한 값이다

방법	20 updates	100 updates	비교 기준
Optimal	679.31	679.31	exhaustive search
HEFT-based	800.75	800.75	heuristic
Fine-tuning DRL	812.32	789.92	pretrained 뒤 새 task update
MRLCO	791.03	722.63	meta 초기값 뒤 새 task update

Topology I에서 MRLCO는 fine-tuning DRL보다 낮지만, table의 optimal latency보다는 높다

일반 DRL·DMRO·MRLCO의 문제 설정 차이

구분	일반 DRL	DMRO	MRLCO
placement	source마다 다름	local, edge, cloud	UE local 또는 MEC
inner learner	한 맥락의 RL policy	parallel Q-learning DNN	PPO policy/value seq2seq
meta 역할	보통 없음	환경 간 초기값 ψ	MDP distribution의 초기값 θ
구조	문제별 상이	dependent workflow sequence	DAG와 dependency 순서
근거 범위	각 source 조건	plot 기반 portability	held-out adaptation plots와 Table 3

MRLCO를 DMRO Q-learning이나 단순 pretraining과 혼동하면 안 된다

논문이 남긴 가정과 한계

발표의 결론은 meta-RL의 가능성이지 모든 MEC 환경에서의 무조건적 안정성이나 최적성은 아니다

원문 가정 또는 한계

stable wireless channel, reliable UE, sufficient resource

synchronous outer update

synthetic DAG simulator

first-order approximation

왜 결과 해석에 중요한가

모바일 네트워크의 변동과 고장, 자원 부족을 직접 검증하지 않는다

모든 UE parameter를 기다리면 straggler가 training을 지연시킬 수 있다

실제 application trace와 hardware runtime을 측정한 결과는 아니다

second-order MAML 비용을 줄이지만 approximation trade-off를 둔다

MRLCO는 빠른 적응을 보였지만, scale과 straggler 문제는 논문이 후속 과제로 남긴 영역이다

MRLCO의 핵심 연결

- | | | |
|----|---------------------|---|
| 01 | DAG 상태 | dependency와 partial plan을 policy 입력으로 만든다 |
| ↓ | | |
| 02 | seq2seq scheduling | parent를 앞에 두고 local/offload action을 생성한다 |
| ↓ | | |
| 03 | inner adaptation | 새 MEC context에서 PPO로 θ_i 를 조정한다 |
| ↓ | | |
| 04 | outer meta-learning | 여러 context의 적응 결과로 다음 θ 를 개선한다 |

DAG-aware policy와 meta 초기값으로 새 context 적응을 빠르게 한다